

Your Remote Arm - Research Phase

Aaron Rhim, Felix Perron, Nathaniel Hawron, Zachrey Zhu, Leo Kaiya, Cohen, and Axel Bendl

University of British Columbia, British Columbia, Canada
rhimaaron@gmail.com

1 Mechanical Hand Design

1.1 Considerations

We must decide whether to have a two-finger grip or a multi-finger grip. The neck will most likely be bought off the shelf. This may include having a pre-installed grip. The purpose of buying an arm rather than creating one is to highlight the purpose of the project, which is to optimize the reconstruction of a video through computer vision and control sensors. For this, I want to avoid going through the problem of getting the arm to work in the first place, in comparison to having a functional arm and getting the optimization to work.

1.2 Two-Finger Grip Configuration

A "parallel gripper" consists of just two fingers and are the industry standard for highly repetitive, simple tasks.

Advantages A Two-Finger Gripper is valuable because of the simplicity of design. There are fewer moving parts, and often few actuators which will reduce the overall complexity and cost of the arm. This also means it is more reliable and robust in terms of consistency which is easier to calibrate, has fewer points of failure, and will be better at maintaining a consistent performance. The control algorithm is also much more straightforward (depending on the amount of actuators) and this model could open the path to optimizing response time in which the AI model replicates the task.

Disadvantages As a result of this simplicity, the arm will struggle with more dexterous tasks that demand more from the in-hand orientation. There would also be restrictions in the hand's grasping versatility. Does not easily adapt to an object's contours or orientations. Also, depending on the material the claw is made up of, the two-point contact may result in slippage when handling objects. Force distribution would also be very difficult to maintain.

1.3 Multi-Finger Grip Configuration

May be in the form of a human-hand (highlight on multiple joints in addition to multiple fingers).

Advantages A hand would be more dexterous and versatile. This allows for in-hand manipulation. There would be an improved force distribution - more contact points. It would also be more adaptable to diverse tasks which allows for a closer resemblance to a human hand (aesthetic considerations). Aligns closer to project goals. Additionally, this would allow us to use pose landmarks which better aligns with a structure in the form of a human hand.

Disadvantages Multi-finger grippers usually have many joints (often in the fingers) which increases the mechanical complexity of the build and this would also require us to design and assembling an adaptable gripper that can supposedly attach to the existing arm that we have. There would also be complex and sophisticated control requirements to ensure finger synchronization and a customizable force distribution to the object in frame. This would also cost more (mainly for the actuators) and the development time would undoubtedly take longer.

Gripper Design	Pros (Capabilities & Ease)	Cons (Complexity & Limits)	Use Cases
2-Finger Parallel	Simple, low-cost (1-2 motors) Robust & reliable Grasps simple shapes Easy control (open/close)	Can't reorient objects in hand Unstable for large round objects without a 3rd contact Limited adaptability to odd shapes	General pick-and-place, most objects. Best when simplicity and reliability are paramount
3-Finger Adaptive	More stable grasp on spheres/cylinders Can perform in-hand rotation (tripod grip)	Higher cost and control complexity Moderate calibration needed for finger synchronization	Precision holds (e.g. a ball), manipulating orientation of held object. Common in research (e.g. used in DARPA Challenge robots)
5-Finger Hand	Closest to human hand versatility (grasp nearly any shape) Visually impressive, dexterous (can do fine tasks like tool use)	High complexity: many DOFs to control, difficult to tune Expensive if bought (up to tens of thousands) vs. time-consuming if custom Hard to fully utilize within one summer timeline	Advanced manipulation (e.g. human-like tasks). Great for demonstration impact, but risky for a short project unless using existing designs.

1.4 Off-The-Shelf Arm vs. Custom Build

We must decide whether to use an off-the-shelf arm or build an arm from scratch. I have already mentioned the majority of the reason why I believe we should buy a pre-made arm and change the claw itself however, I want your input too. Additionally, whatever choice we decide on must allow us to design our own gripper.

Off-The-Shelf Arm We would be reusing a pre-made model of an arm that provides a tested, reliable platform for the manipulator. One downside could be that in presenting this project, the arm may seem less exciting because the model is not unique, however, we are already designing the gripper ourselves and so, anymore designer may prove to be more of a setback.

Custom Build There are ways to make the custom build idea to work. In one summer, I propose a compromise: start with an open-source arm design (or kit) and then upgrade the parts ourselves as time allows. This way, we have control over the design (which would be very rudimentary) while still having a functional model with built-in servo integration software (ideally).

2 Project Goal Definition

Our goal for this summer is to create a robotic system that can mimic human tasks through vision-driven control. It is not our goal to create the most advanced hand which means this project will be measured by the success and interactivity of the system. Essentially, how well the robot will reproduces actions. Additionally, keeping the setup low-cost and accessible is important so that the project can be replicated by others and so that the model can be trained for future purposes.

2.1 Software over Hardware

We must emphasize the optimization of the software and computer vision rather than the hardware. What this also implies is that we cannot spend time optimizing the claw structure to do a single specific task. We must first find success in replicating simple tasks and move to harder tasks as time allows. This would also allow us to test how well inexpensive robots can learn complex manipulation. Additionally, we could train the model iteratively and if it cannot reproduce an action right away, we could use a trial-and-error design.

2.2 Priority Order

- **Visual Perception:** Develop a vision system both constant and dynamic. We will test the model first in a stable frame where the task being replicated is always a certain distance away. Then, we will see the replication accuracy in using different angles and distances.
- **Control Intelligence:** We must implement computer vision models and algorithms that can translate visual inputs into arm motions.
- **Interactivity:** We must ensure that the robot can be interacted with in real time. This may involve creating prototypes to test
- **Low Cost & Expandability:** The model must be easily manipulated so that certain parts can be swapped out (e.g. the claw).

3 Data Collection

Vision Setup	Components & Cost	Data Captured	Pros	Cons / Trade-offs
Single RGB Camera (monocular)	One webcam or smartphone	2D images (requires depth inference)	Lowest cost, simplest hardware. No special sensors required. Abundant online training data.	Depth ambiguity; complex algorithms needed. Sensitive to lighting and texture.
Stereo Cameras	Two synced cameras + calibration rig	Stereo image pairs for direct depth calculation	Direct geometric depth via triangulation. Mature OpenCV algorithms available.	Requires careful calibration. Heavy processing for high-res real-time maps.
Depth Camera (RGB-D)	Kinect v2 or Intel RealSense	Depth map (plus aligned RGB image)	High quality depth out-of-the-box. SDK support for skeleton tracking.	Advanced sensor; less CV learning. Added cost; compatibility issues.
Fiducial Markers (e.g., ArUco)	Printed markers + single camera	2D images with known marker patterns	Reliable 6DOF pose estimation. Minimal computation. Very low cost.	Unnatural; requires physical markers. Fails if markers occlude.
Wearable Sensors	IMU suit, data gloves, or smartphone on limb	Direct human joint angles/motion	Accurate capture of motion. Facilitates real-time teleoperation.	Extra hardware cost; additional calibration. Moves away from pure vision goals.

3.1 Primary Choice: Monocular RGB Camera

The project specifies an RGB camera (e.g., iPhone camera, webcam) as the main sensor. This adheres to a few of our main goals which is accessibility and low cost. The main challenge will be that the camera provides no depth estimation features so, we must create a system that can infer such estimations automatically. This will most likely be done using a pre-trained model. A couple suggestions for how the iterative design would work are:

1. Regulating distance of the RGB camera from location of interest
2. Successfully convert acquired predictions and replication data into motion
3. Upon success of previous step, create a new algorithm that can infer without a regulated distance from location of interest
4. Repeat step 2

4 Computer Vision Techniques

Assuming we decide to use the monocular camera setup, We must then design a model that can extract a 3D understanding from the feed and process the information to be sent to the motor controls. Key tasks include: depth estimation, object detection/tracking, and pose estimation.

4.1 Monocular Depth Estimation - Baseline and Limitations

	Description	Pros	Cons
DPT Transformer	Similar to an autoencoder, this model includes no training but intense highlight on model architecture. Through mathematical algorithms, the model will achieve a state-of-the-art accuracy.	Accuracy will essentially be the same and will not change because the depth is being created through visual transformation.	Model architecture is PhD-level Deep Learning. Existing models are likely difficult to understand and implement.
U-Net Architecture	Re-uses a single image to train which can be useful but there is a high detail loss. Essentially, the model will encourage neighboring pixels to have similar depth, except where image gradients indicate an edge. This means that if a depth discontinuity has no clear color or intensity change (an “invisible” ledge), the network may incorrectly smooth over it.	CNN baseline and architecture is relatively simple to understand.	Model will overfit in most cases and produce overly smooth depth maps (key component to what makes the U-Net architecture). Difficult to optimize considering the architecture relies on max pooling (already optimized).
AdaBins	State-of-the-art with adaptive binning; excellent for reducing scale ambiguity.	Highly adaptable. With enough training, it can become a very accurate model. May allow a combination of 3D object detection	Requires very specific parameters for where the camera should be placed etc.

4.2 Monocular Depth Estimation - Edge and Occlusion-Aware Depth Refinement

	Description	Pros	Cons
Occlusion Boundary Detection Networks Sources: Journal , Journal	Instead of refining depth directly, another strategy is to train a model to output occlusion edge maps from the RGB image. An occlusion map is a greyscale image, with white indicating areas that should receive full indirect lighting, and black indicating no indirect lighting. Basically, the darker the color, the further back it is.	Earlier works by Huan Fu et al. (CVPR 2016) use CNN+CRF to label occlusion boundaries in images by leveraging context and even temporal cues. These techniques treat occlusion detection as its own task – producing a binary mask of likely depth discontinuities (range). Modern approaches like Neural-CRF Depth Refinement (Yuan et al. 2022) integrate a learned CRF module into the network decoder to enforce sharp depth changes at predicted edges.	Training a dedicated boundary detector needs labeled data (or careful self-supervision) meaning we will have limited objects that the model can analyze and it may require manually going, frame-by-frame, and measuring the depth ourselves (initially).
Edge-Enhanced Depth Networks Sources: Journal	A recent approach, “The Devil is in the Edges” (ECFNet), explicitly feeds edge information into the depth network. It uses a hybrid edge detection module to produce an edge map and an “edge-highlighted” RGB image, runs a base depth estimator on all inputs, then fuses the results in a depth Consistency Fusion module.	By giving the network high-contrast cues of where intensity edges are, it predicts depth with clearer object boundaries and fine details. The authors found that even using a sobel edge map as an input can yield sharper depth maps, and their learned fusion further improved accuracy.	This multi-input approach requires a bit more processing (edge detection + multiple forwards passes), but is still monocular and has shown state-of-the-art performance on public depth benchmarks.

4.3 Monocular Depth Estimation - Temporal Structure-from-Motion Cues

Here we turn a monocular camera into a form of a stereo device in that we derive structure from movement. Depending on how the camera itself moves, we can use that to create our depth estimates. These ideas will involve changing our method of data collection however, it would still be completely feasible.

	Description	Pros	Cons
Self-Supervised Monocular Depth from Video Sources: Journal	Monodepth2, a study that used self-supervised monocular depth, used stereo-pairs (two videos of the same environment, taken at slightly different positions), to estimate depth.	The model chooses the minimum photometric error per pixel across the frames which yields fairly accurate depth maps even when parts of the scene go out of view in the next frame.	Unfortunately, this model assumes the camera is moving and the scene is mostly static (we want to do the opposite).
Motion-Based Drop-off Detection Sources: Journal	One classical algorithm explicitly tracks the apparent motion of the ground plane versus the motion of the background to find occluding edges of drop-offs.	This is the most classic and simplest design to infer temporal structure.	This design doesn’t give a full depth map, but it can directly signal a cliff edge in front of the robot even if the area lacks texture. Again, this design will require the camera to be moving.

4.4 Monocular Depth Estimation - Depth + Segmentation/Recognition

	Description	Pros	Cons
Depth + Semantic Segmentation Sources: Journal	This idea involves creating different models depending on the type of object in the frame. For a robotic scenario, one could fine-tune a segmentation model to label “floor” vs “void” (drop) vs “obstacle” and use that along with depth.	There are existing, available datasets that have some form of semantic labeling however, most are created to be used in a large, robotic environment.	The the need for segmentation training data and the extra runtime of a segmentation branch is a pain, but efficient architectures exist and the tasks can run in parallel.
Uncertainty Estimation Sources: Journal , Journal , Journal	The idea is that many depth models can be used to predict an uncertainty map for depth. We know that uncertainty is higher at edges and for distant untextured areas. For instance, if the floor ahead suddenly has very high depth uncertainty, that could be a signal of a possible drop-off – prompting the robot to slow down or use an alternate sensor (or a probing action).	With the right probing responses and “checking” designs, this method could prove to be the most accurate of all models.	This design would require a lot of interaction from the user, outside of the simple recording, which removes the possibility to use online videos as testing data.

4.5 Object and Pose Detection

- Use libraries such as OpenPose or MediaPipe for human keypoints.
- For object detection, consider pretrained COCO models (e.g., YOLO or SSD).

4.6 Fusion and Integration

The secret sauce is in fusing these outputs to command the arm. The system must be robust against unstable keypoint detections or noisy depth estimates. Early experiments should guide how to tune and filter these outputs.

5 Team Management Structure

With a 9-person team and a short timeline, clear organization and role allocation is vital. The project spans multiple disciplines, so assigning sub-teams based on expertise is key.

5.1 Role Assignment and Sub-teams

- **Project Manager / Systems Lead:** Oversee progress, integration, and communication.
- **Mechanical Team (2 people):** Responsible for hand and arm design.
- **Electrical/Controls Team (1–2 people):** Set up electronics, microcontrollers, and low-level control.
- **Computer Vision & AI Team (3 people):** Focus on depth estimation, object/pose detection, and software integration.
- **Integration & Testing Team (1–2 people):** Continuously test subsystems and the complete system.
- **Documentation & Presentation Lead (1 person):** Ensure thorough documentation and create final presentation materials.

5.2 Development Methodology (Sprints)

1. **Weeks 1–2:** Planning and onboarding — finalizing requirements, initial feasibility tests, and setting up communication tools.
2. **Weeks 3–4:** Prototype sprint — basic arm assembly, preliminary vision pipeline, and early integration tests.
3. **Weeks 5–7:** Integration Sprint 1 — connect vision with arm motion; refine control algorithms.
4. **Weeks 8–9:** Refinement Sprint 2 — improve robustness, optimize speed, and prepare the demo narrative.
5. **Week 10:** Testing and Demo Prep — full rehearsals, final adjustments, and contingency planning.

5.3 Team Coordination Table

Table 1: Team Roles and Key Tasks

Team Member	Primary Role	Key Tasks
Aaron (DS)	Project Manager, AI Lead	Overall coordination; integration testing; documentation oversight.
Felix (EE)		
Nathaniel (EE)		
Zachrey (Mech)		
Marc (CE)		
Leo (CS)		
Cohen (CS)		
Axel (EE)		

6 Competitions and Existing Projects

It is wise to learn from others who have tackled similar problems. Relevant competitions and open-source projects include:

6.1 Relevant Competitions

- **RoboCup@Home and ARM Challenge:** Focus on vision-guided object manipulation.
- **DIY Robotic Competitions / Hackathons:** E.g., Hackaday Prize, Cornell Cup.
- **FIRST/FTC Robots:** Provide practical tips on robust, real-time vision on limited hardware.

6.2 Existing Open-Source Projects

- **Braccio CamAI (GitHub):** Example of ROS integration with a low-cost robotic arm and computer vision.
- **InMoov Hand Project:** For mechanical reference on multi-finger designs.
- **ROS MoveIt/Pick-and-Place demos:** For learning motion planning and integration.
- **Imitation Learning projects:** E.g., end-to-end learning from demonstration.

6.3 Table 3: Similar Projects & Features for Adaptation

Table 2: Existing Projects and Their Key Features

Project / Repo	Key Features	Reuse/Adaptation
RoboCup ARM Challenge	Vision-based object sorting; standard objects (bottles/cans).	Use as benchmark; adapt similar pick-and-place tasks.
Braccio CamAI (GitHub)	Low-cost 5DOF arm; ROS integration; object detection via TensorFlow.	Reference ROS configuration; adapt vision nodes for object detection.
OpenCV Gesture Arm (GitHub)	OpenCV-based gesture recognition to control a 5DOF arm.	Learn mapping of image features to servo commands; simpler gesture control.
InMoov Hand Project	3D-printed humanoid hand with community support.	Use mechanical designs for multi-finger prototypes if desired.
AdaBins Depth (GitHub)	State-of-art depth model with adaptive binning.	Use pretrained weights for accurate depth estimation.
MediaPipe Hands (Google)	Real-time 21-keypoint hand tracking on CPU.	Use for human pose estimation and grasp intent detection.

7 Conclusion

By leveraging pretrained CV models and existing open-source projects, the team can focus on integrating an affordable hand/arm with smart vision. An organized team structure and clear development milestones will ensure both successful execution and valuable learning. This approach aims to yield a functional demo with robust vision-driven control while keeping the system low-cost and expandable for future work.

References